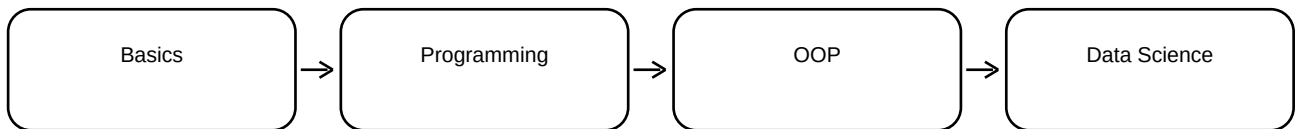


PYTHON PROGRAMMING

Complete Beginner-Friendly Study Material

Learn Python Step by Step



Easy English • Examples • Diagrams • Practice

Prepared as a student study guide

TABLE OF CONTENTS

1. 1. What is Python?
2. 2. Python Features and Applications
3. 3. Installing Python and First Program
4. 4. Python Syntax and Comments
5. 5. Variables and Data Types
6. 6. Input and Output
7. 7. Operators
8. 8. Conditional Statements
9. 9. Loops
10. 10. Strings
11. 11. Lists
12. 12. Tuples
13. 13. Sets
14. 14. Dictionaries
15. 15. Functions
16. 16. Recursion
17. 17. Modules and Packages
18. 18. Exception Handling
19. 19. File Handling
20. 20. Object-Oriented Programming
21. 21. Classes and Objects
22. 22. Inheritance
23. 23. Polymorphism
24. 24. Encapsulation
25. 25. Abstraction
26. 26. NumPy Basics
27. 27. Pandas Basics
28. 28. Matplotlib Basics
29. 29. Python for Data Science
30. 30. Mini Projects

31. 31. Debugging and Good Practices

32. 32. Common Interview Questions

33. 33. Practice Programs

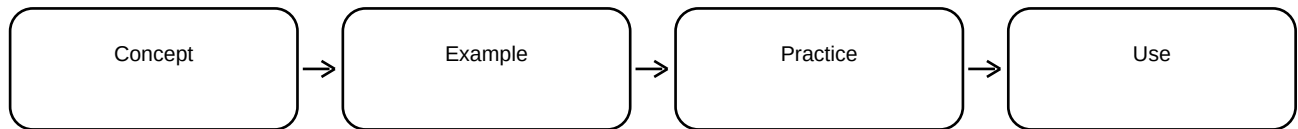
34. 34. Final Revision

Note: This material uses simple English. Read one chapter at a time and practice every program.

Chapter 1: What is Python?

Python is a high-level, general-purpose programming language designed to make programming readable.

What is Python?



Simple meaning

Python lets us give instructions to a computer using simple English-like syntax.

Python was created by Guido van Rossum and first released in 1991.

Uses

- Web development
- Data science
- Automation
- Artificial intelligence
- Testing

Example Program

```
print('Hello, Python!')
```

Output: Hello, Python!

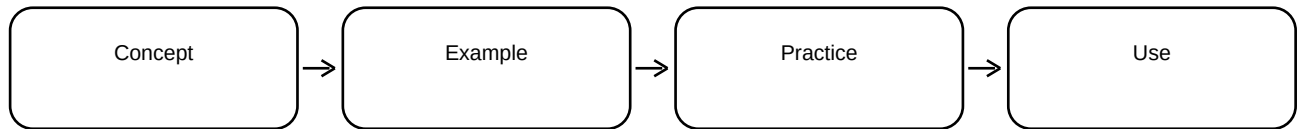
Practice Questions

- Define What is Python?.
- Write one simple example of What is Python?.
- Give one practical use of What is Python?.

Chapter 2: Python Features and Applications

Python is easy to learn and has a large ecosystem.

Python Features and Applications



Features

- Readable syntax
- Interpreted
- Dynamically typed
- Object-oriented support
- Cross-platform

Applications

Python is used in web apps, automation, data analysis, AI, testing, and education.

Example Program

```
name = 'Bhavya'
print(name)
```

Output: Bhavya

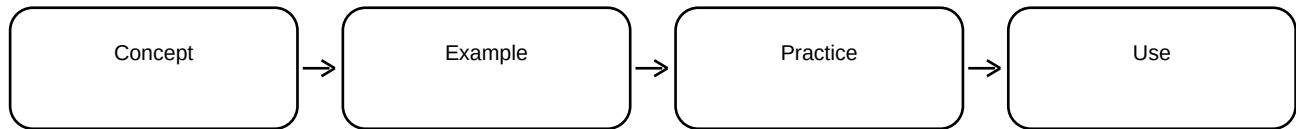
Practice Questions

- Define Python Features and Applications.
- Write one simple example of Python Features and Applications.
- Give one practical use of Python Features and Applications.

Chapter 3: Installing Python and First Program

The Python interpreter reads and runs Python code.

Installing Python and First Program



First program

```
print('Hello World')
```

The print() function displays information.

Variables

```
name = 'Bhavya'
print('Hello', name)
```

Example Program

```
print('Welcome to Python')
```

Output: Welcome to Python

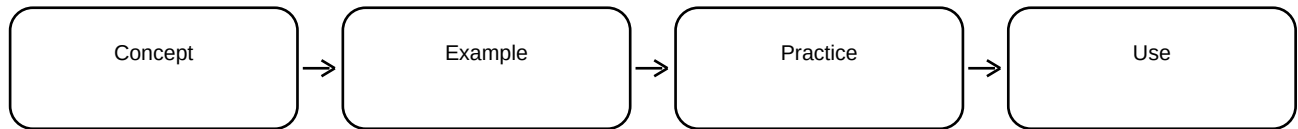
Practice Questions

- Define Installing Python and First Program.
- Write one simple example of Installing Python and First Program.
- Give one practical use of Installing Python and First Program.

Chapter 4: Python Syntax and Comments

Syntax means the rules used to write Python programs.

Python Syntax and Comments



Indentation

Python uses indentation to define blocks. Four spaces are commonly used.

Comments

```
# This is a comment  
print('Python')
```

Identifiers

Names can contain letters, digits, and underscore, but cannot start with a digit.

Example Program

```
if 10 > 5:  
    print('10 is greater')
```

Output: 10 is greater

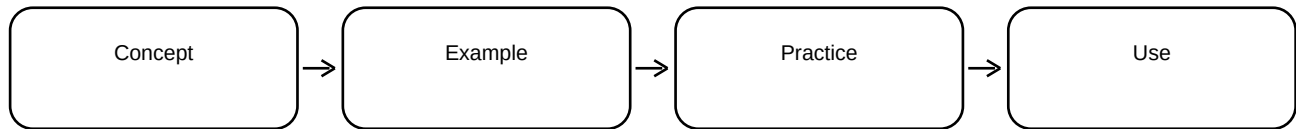
Practice Questions

- Define Python Syntax and Comments.
- Write one simple example of Python Syntax and Comments.
- Give one practical use of Python Syntax and Comments.

Chapter 5: Variables and Data Types

Variables refer to values, and Python determines their types at runtime.

Variables and Data Types



Common types

- int
- float
- str
- bool
- list
- tuple
- set
- dict

Type conversion

```
x = '25'  
y = int(x)  
print(y + 5)
```

Example Program

```
marks = 85  
print(type(marks))
```

Output:

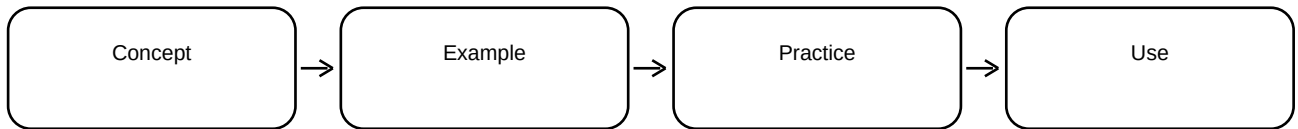
Practice Questions

- Define Variables and Data Types.
- Write one simple example of Variables and Data Types.
- Give one practical use of Variables and Data Types.

Chapter 6: Input and Output

input() gets keyboard input and print() displays output.

Input and Output



Input

```
name = input('Enter your name: ')
print('Hello', name)
```

Numeric input

```
age = int(input('Enter age: '))
print(age + 1)
```

Formatted output

```
name='Bhavya'
score=90
print(f'{name} scored {score}')
```

Example Program

```
a = int(input('Enter a: '))
b = int(input('Enter b: '))
print(a + b)
```

Output: The sum of the two numbers

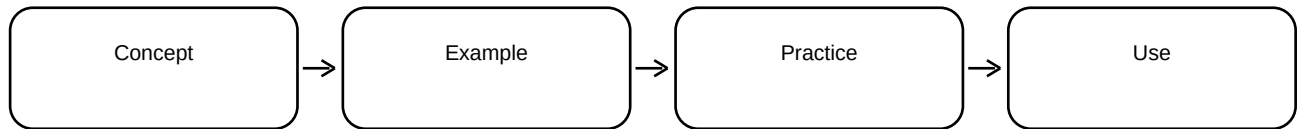
Practice Questions

- Define Input and Output.
- Write one simple example of Input and Output.
- Give one practical use of Input and Output.

Chapter 7: Operators

Operators perform calculations and comparisons.

Operators



Arithmetic

- +, -, *, /, //, %, **

Comparison

- ==, !=, >, <, >=, <=

Logical

- and
- or
- not

Example Program

```
a=10
b=3
print(a//b)
print(a%b)
```

Output: 3 and 1

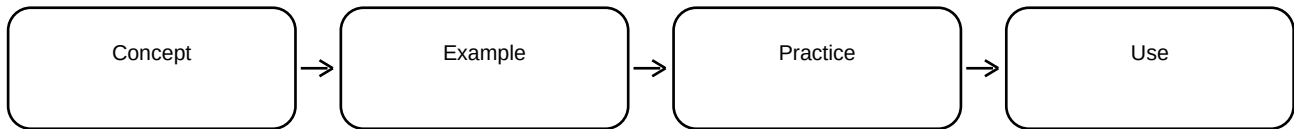
Practice Questions

- Define Operators.
- Write one simple example of Operators.
- Give one practical use of Operators.

Chapter 8: Conditional Statements

Conditions allow a program to make decisions.

Conditional Statements



if

```
age=18
if age>=18:
    print('Eligible')
```

if-else

```
n=7
if n%2==0:
    print('Even')
else:
    print('Odd')
```

elif

Use elif for multiple possible conditions.

Example Program

```
marks=75
if marks>=90:
    grade='A'
elif marks>=60:
    grade='B'
else:
    grade='C'
print(grade)
```

Output: B

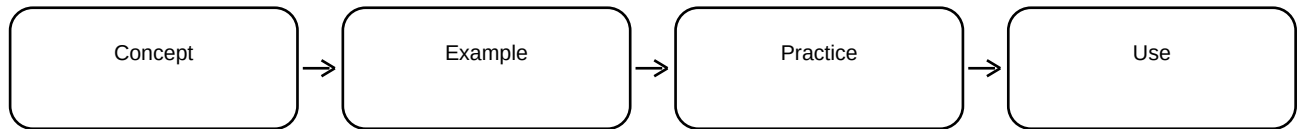
Practice Questions

- Define Conditional Statements.
- Write one simple example of Conditional Statements.
- Give one practical use of Conditional Statements.

Chapter 9: Loops

Loops repeat code. Python mainly provides for and while.

Loops



for

```
for i in range(1,6):  
    print(i)
```

while

```
i=1  
while i<=5:  
    print(i)  
    i+=1
```

break and continue

break stops a loop; continue skips to the next iteration.

Example Program

```
total=0  
for i in range(1,6):  
    total += i  
print(total)
```

Output: 15

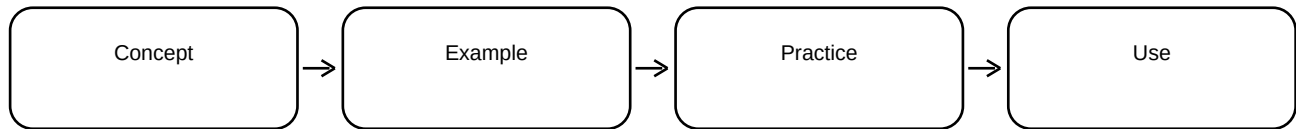
Practice Questions

- Define Loops.
- Write one simple example of Loops.
- Give one practical use of Loops.

Chapter 10: Strings

A string is a sequence of characters.

Strings



Indexing

```
text='Python'  
print(text[0])  
print(text[-1])
```

Slicing

```
print(text[0:3])
```

Methods

- upper()
- lower()
- strip()
- replace()
- split()
- join()

Example Program

```
text='python programming'  
print(text.upper())
```

Output: PYTHON PROGRAMMING

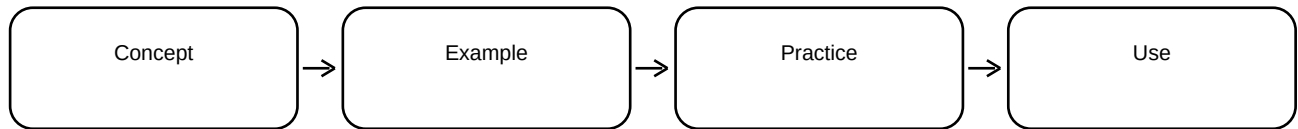
Practice Questions

- Define Strings.
- Write one simple example of Strings.
- Give one practical use of Strings.

Chapter 11: Lists

Lists store ordered, changeable collections.

Lists



Methods

- `append()`
- `extend()`
- `insert()`
- `remove()`
- `pop()`
- `sort()`

List comprehension

```
squares=[x*x for x in range(1,6)]  
print(squares)
```

Example Program

```
numbers=[1,2,3,4]  
numbers.append(5)  
print(numbers)
```

Output: [1, 2, 3, 4, 5]

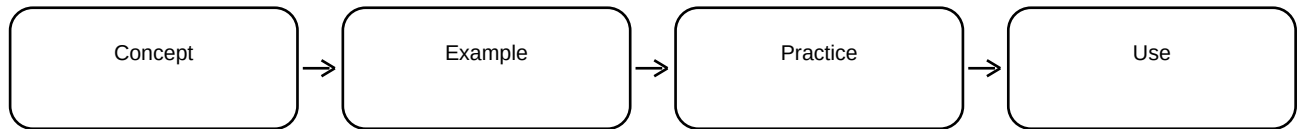
Practice Questions

- Define Lists.
- Write one simple example of Lists.
- Give one practical use of Lists.

Chapter 12: Tuples

Tuples are ordered collections that are normally immutable.

Tuples



Creating

```
point=(10, 20)
print(point)
```

Unpacking

```
x,y=point
print(x,y)
```

Use

Tuples are useful for fixed groups of related values.

Example Program

```
data=( 'Python', 3, True)
print(data[0])
```

Output: Python

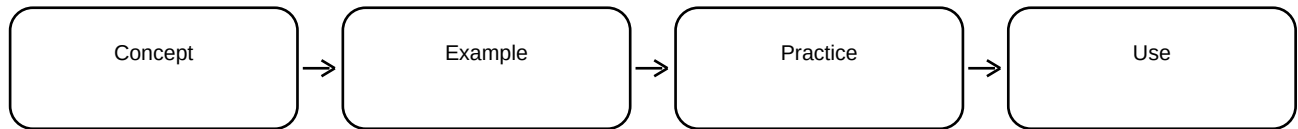
Practice Questions

- Define Tuples.
- Write one simple example of Tuples.
- Give one practical use of Tuples.

Chapter 13: Sets

Sets store unique values and support mathematical set operations.

Sets



Operations

- union
- intersection
- difference

Example

```
a={1, 2, 3}
b={3, 4, 5}
print(a & b)
```

Example Program

```
a={1, 2, 3}
b={3, 4, 5}
print(a & b)
```

Output: {3}

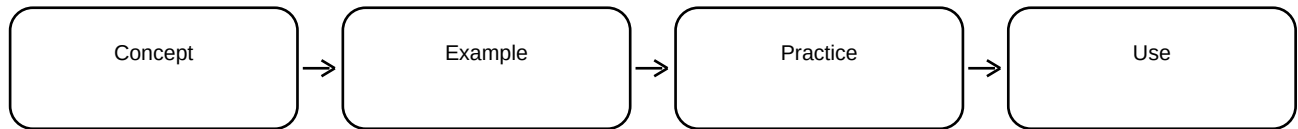
Practice Questions

- Define Sets.
- Write one simple example of Sets.
- Give one practical use of Sets.

Chapter 14: Dictionaries

Dictionaries store key-value pairs.

Dictionaries



Example

```
student={'name': 'Bhavya', 'age': 23}  
print(student['name'])
```

Methods

- get()
- keys()
- values()
- items()
- update()

Example Program

```
marks={'Math': 90, 'Python': 95}  
print(marks['Python'])
```

Output: 95

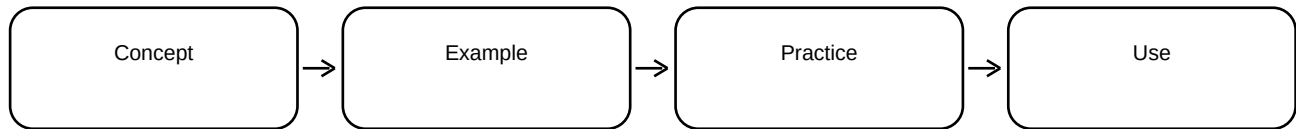
Practice Questions

- Define Dictionaries.
- Write one simple example of Dictionaries.
- Give one practical use of Dictionaries.

Chapter 15: Functions

Functions are reusable blocks of code.

Functions



Definition

```
def add(a,b):  
    return a+b  
print(add(10,20))
```

Parameters

Parameters are names in a definition; arguments are values passed to a call.

Default arguments

```
def power(x,n=2):  
    return x**n
```

Example Program

```
def add(a,b):  
    return a+b  
print(add(10,20))
```

Output: 30

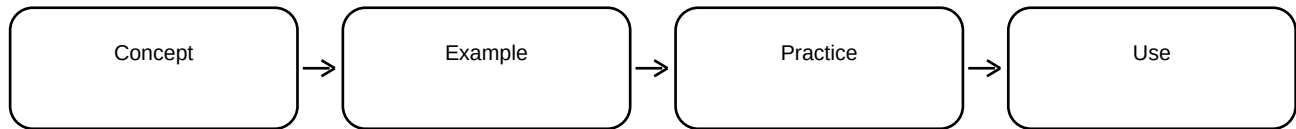
Practice Questions

- Define Functions.
- Write one simple example of Functions.
- Give one practical use of Functions.

Chapter 16: Recursion

Recursion happens when a function calls itself.

Recursion



Factorial

```
def fact(n):  
    if n==0:  
        return 1  
    return n*fact(n-1)
```

Base case

Every recursive function needs a condition that stops further calls.

Example Program

```
def countdown(n):  
    if n==0:  
        print('Done'); return  
    print(n)  
    countdown(n-1)  
countdown(3)
```

Output: 3, 2, 1, Done

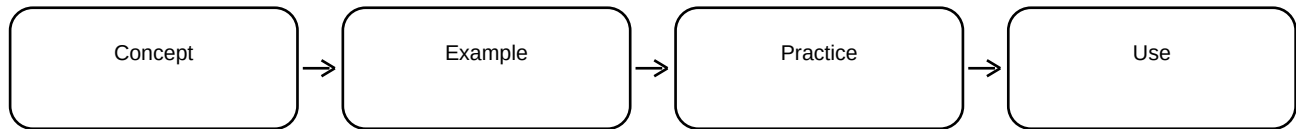
Practice Questions

- Define Recursion.
- Write one simple example of Recursion.
- Give one practical use of Recursion.

Chapter 17: Modules and Packages

Modules contain reusable Python code; packages organize related modules.

Modules and Packages



Import

```
import math
print(math.sqrt(25))
```

Useful modules

- math
- random
- datetime
- os

Example Program

```
import math
print(math.pi)
```

Output: 3.141592653589793

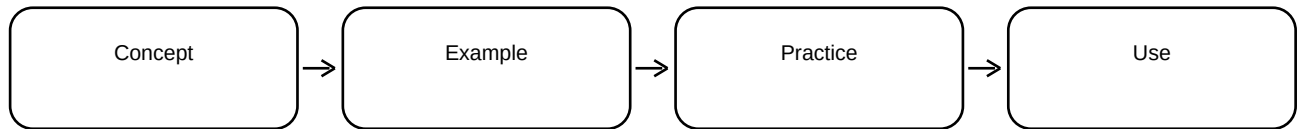
Practice Questions

- Define Modules and Packages.
- Write one simple example of Modules and Packages.
- Give one practical use of Modules and Packages.

Chapter 18: Exception Handling

Exceptions are runtime problems that can be handled with try and except.

Exception Handling



try-except

```
try:
    print(10/0)
except ZeroDivisionError:
    print('Cannot divide by zero')
```

finally

finally runs whether an exception happens or not.

raise

```
raise ValueError('Invalid value')
```

Example Program

```
try:
    print(10/0)
except ZeroDivisionError:
    print('Cannot divide by zero')
```

Output: Cannot divide by zero

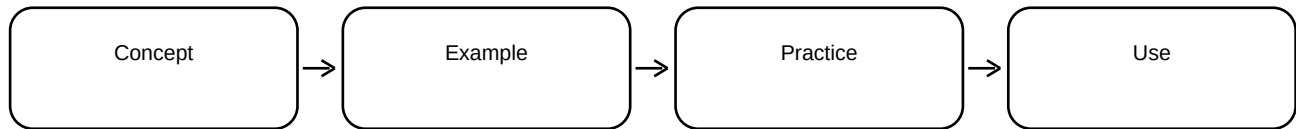
Practice Questions

- Define Exception Handling.
- Write one simple example of Exception Handling.
- Give one practical use of Exception Handling.

Chapter 19: File Handling

File handling lets Python read and write stored data.

File Handling



with

```
with open('notes.txt', 'w') as f:  
    f.write('Hello Python')
```

Modes

- r read
- w write
- a append
- b binary

Example Program

```
with open('sample.txt', 'w') as f:  
    f.write('Hello Python')
```

Output: A text file is written.

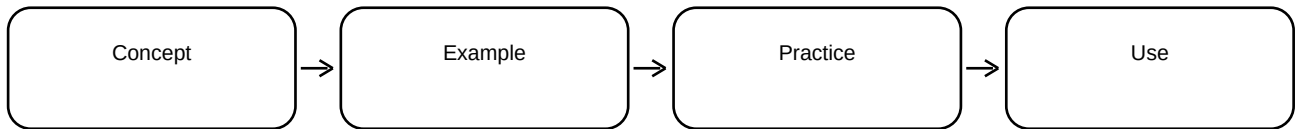
Practice Questions

- Define File Handling.
- Write one simple example of File Handling.
- Give one practical use of File Handling.

Chapter 20: Object-Oriented Programming

OOP organizes programs around objects containing data and behavior.

Object-Oriented Programming



Main ideas

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Class and object

A class is a blueprint; an object is an instance of that blueprint.

Example Program

```
class Student:
    def greet(self):
        print('Hello')
Student().greet()
```

Output: Hello

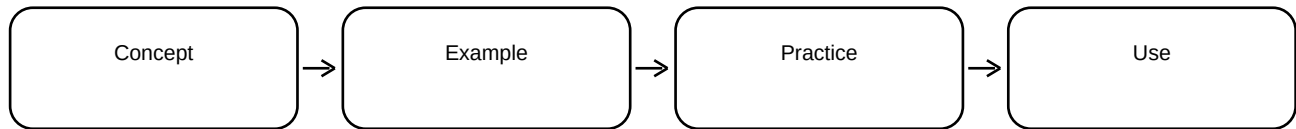
Practice Questions

- Define Object-Oriented Programming.
- Write one simple example of Object-Oriented Programming.
- Give one practical use of Object-Oriented Programming.

Chapter 21: Classes and Objects

Classes define attributes and methods; objects use them.

Classes and Objects



Constructor

```
class Student:
    def __init__(self, name):
        self.name=name
```

self

self refers to the current object.

Methods

Methods are functions inside a class.

Example Program

```
class Car:
    def __init__(self, brand):
        self.brand=brand
car=Car('Python Motors')
print(car.brand)
```

Output: Python Motors

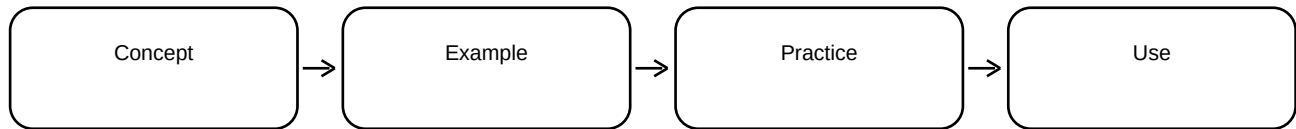
Practice Questions

- Define Classes and Objects.
- Write one simple example of Classes and Objects.
- Give one practical use of Classes and Objects.

Chapter 22: Inheritance

Inheritance allows a child class to reuse or extend a parent class.

Inheritance



Example

```
class Animal:
    def eat(self): print('Eating')
class Dog(Animal): pass
```

Overriding

A child can define a method with the same name as the parent method.

Example Program

```
class Animal:
    def eat(self): print('Eating')
class Dog(Animal): pass
Dog().eat()
```

Output: Eating

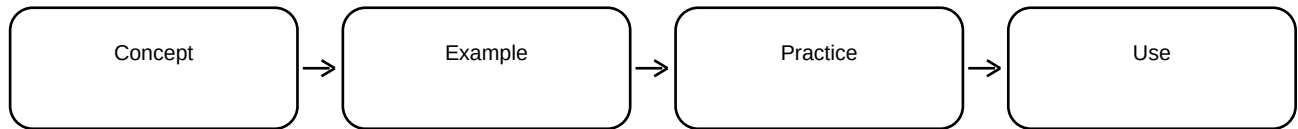
Practice Questions

- Define Inheritance.
- Write one simple example of Inheritance.
- Give one practical use of Inheritance.

Chapter 23: Polymorphism

Polymorphism allows a common operation to behave differently for different objects.

Polymorphism



Example

```
class Dog:
    def sound(self): return 'Bark'
class Cat:
    def sound(self): return 'Meow'
for a in [Dog(),Cat()]: print(a.sound())
```

Example Program

```
class Dog:
    def sound(self): return 'Bark'
class Cat:
    def sound(self): return 'Meow'
for a in [Dog(),Cat()]: print(a.sound())
```

Output: Bark / Meow

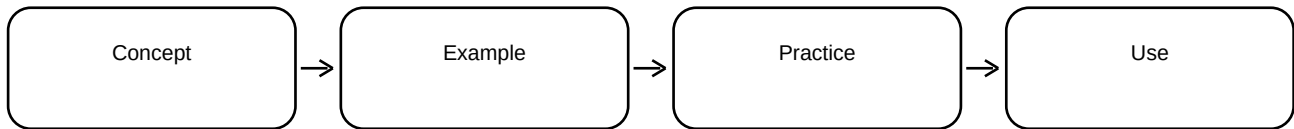
Practice Questions

- Define Polymorphism.
- Write one simple example of Polymorphism.
- Give one practical use of Polymorphism.

Chapter 24: Encapsulation

Encapsulation keeps data and methods together and can provide controlled access.

Encapsulation



Conventions

A leading underscore commonly signals internal-use attributes. Properties can provide controlled access.

Example

```
class Account:
    def __init__(self, balance): self._balance=balance
```

Example Program

```
class Student:
    def __init__(self, marks): self._marks=marks
s=Student(90)
print(s._marks)
```

Output: 90

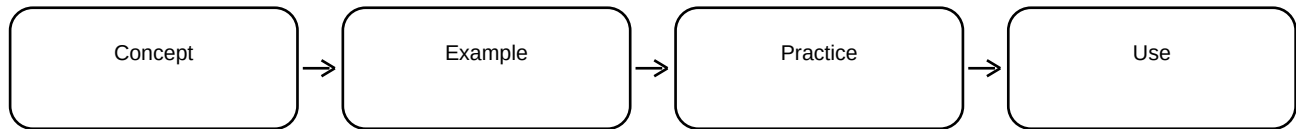
Practice Questions

- Define Encapsulation.
- Write one simple example of Encapsulation.
- Give one practical use of Encapsulation.

Chapter 25: Abstraction

Abstraction shows an important interface while hiding unnecessary implementation details.

Abstraction



ABC

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
```

Use

It helps define what related classes must do without exposing every internal step.

Example Program

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
```

Output: Defines an abstract interface.

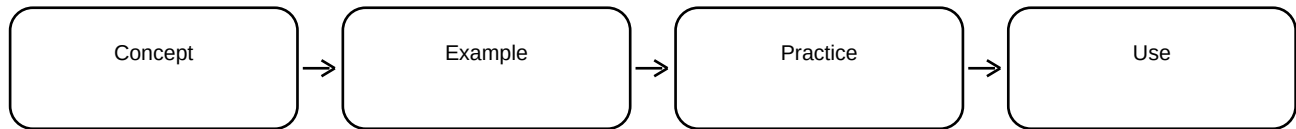
Practice Questions

- Define Abstraction.
- Write one simple example of Abstraction.
- Give one practical use of Abstraction.

Chapter 26: NumPy Basics

NumPy provides efficient arrays and numerical operations.

NumPy Basics



Array

```
import numpy as np
arr=np.array([1,2,3])
print(arr+10)
```

Useful ideas

- shape
- ndim
- reshape
- sum
- mean

Example Program

```
import numpy as np
arr=np.array([1,2,3])
print(arr+10)
```

Output: [11 12 13]

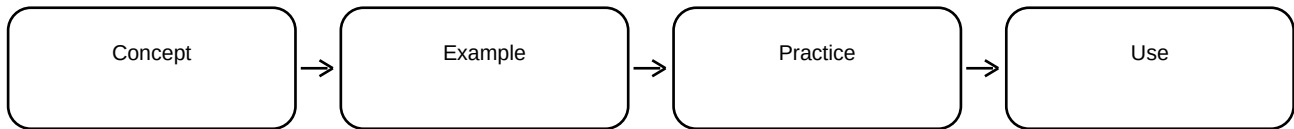
Practice Questions

- Define NumPy Basics.
- Write one simple example of NumPy Basics.
- Give one practical use of NumPy Basics.

Chapter 27: Pandas Basics

Pandas is used for structured data such as tables.

Pandas Basics



DataFrame

```
import pandas as pd
df=pd.DataFrame({'Name':['A', 'B'], 'Marks':[80,90]})
print(df)
```

Methods

- head()
- info()
- describe()
- dropna()
- fillna()
- groupby()

Example Program

```
import pandas as pd
df=pd.DataFrame({'Marks':[70,80,90,60]})
print(df['Marks'].mean())
```

Output: 75.0

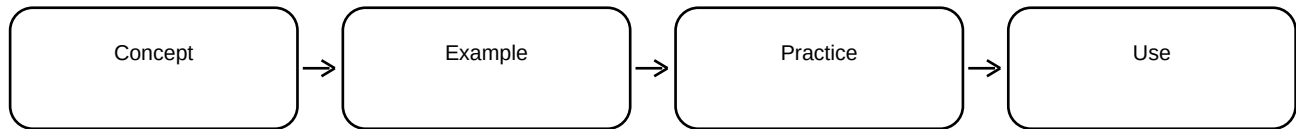
Practice Questions

- Define Pandas Basics.
- Write one simple example of Pandas Basics.
- Give one practical use of Pandas Basics.

Chapter 28: Matplotlib Basics

Matplotlib creates charts and graphs.

Matplotlib Basics



Bar chart

```
import matplotlib.pyplot as plt
names=['A', 'B', 'C']
marks=[70, 85, 90]
plt.bar(names, marks)
plt.show()
```

Chart types

- line
- bar
- scatter
- histogram

Example Program

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
```

Output: A line chart is displayed.

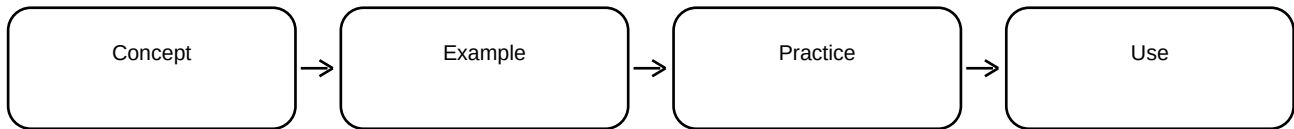
Practice Questions

- Define Matplotlib Basics.
- Write one simple example of Matplotlib Basics.
- Give one practical use of Matplotlib Basics.

Chapter 29: Python for Data Science

Python combines programming with libraries for data analysis and visualization.

Python for Data Science



Workflow

- Collect data
- Load data
- Clean data
- Explore
- Visualize
- Model when needed

Libraries

NumPy, Pandas, Matplotlib, and machine-learning libraries are common tools.

Example Program

```
import pandas as pd
df=pd.DataFrame({'Marks':[70,80,90,60]})
print(df['Marks'].mean())
```

Output: 75.0

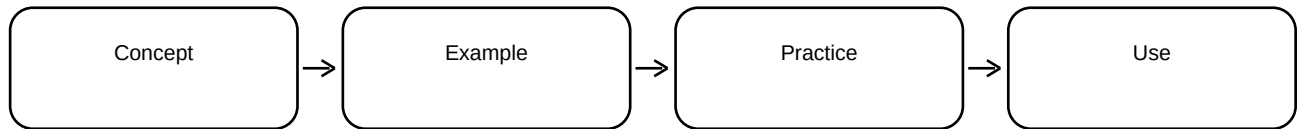
Practice Questions

- Define Python for Data Science.
- Write one simple example of Python for Data Science.
- Give one practical use of Python for Data Science.

Chapter 30: Mini Projects

Projects convert theory into practical skill.

Mini Projects



Ideas

- Calculator
- Student marks manager
- To-do list
- Weather dashboard

Cycle

Problem → requirements → design → code → test → improve → document.

Example Program

```
tasks=[]
tasks.append('Learn Python')
print(tasks)
```

Output: ['Learn Python']

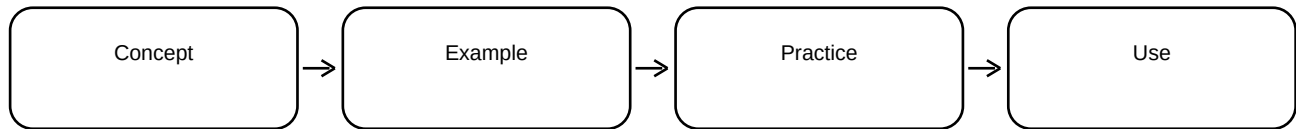
Practice Questions

- Define Mini Projects.
- Write one simple example of Mini Projects.
- Give one practical use of Mini Projects.

Chapter 31: Debugging and Good Practices

Debugging means finding and fixing problems.

Debugging and Good Practices



Common errors

- SyntaxError
- NameError
- TypeError
- ValueError
- IndexError
- KeyError

Good practices

- Meaningful names
- Small functions
- Avoid repetition
- Test edge cases

Example Program

```
numbers=[1, 2, 3]
print(numbers[2])
```

Output: 3

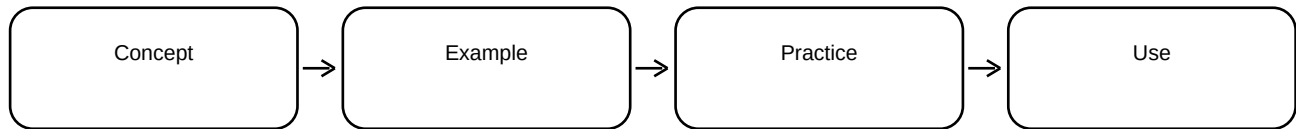
Practice Questions

- Define Debugging and Good Practices.
- Write one simple example of Debugging and Good Practices.
- Give one practical use of Debugging and Good Practices.

Chapter 32: Common Interview Questions

Use these questions for viva and beginner interviews.

Common Interview Questions



Questions

What is Python? What is an interpreter? What is a variable? What is a function? How do list and tuple differ? What is a DataFrame?

Answer method

Give a short definition, a simple example, and one use.

Example Program

```
def add(a,b):  
    return a+b  
print(add(2,3))
```

Output: 5

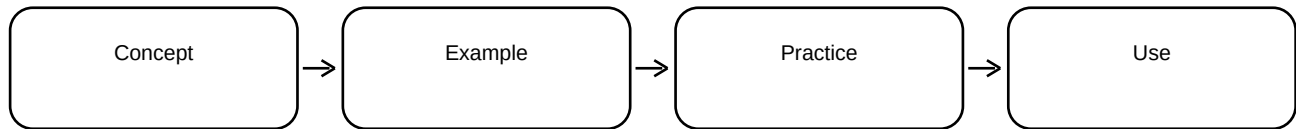
Practice Questions

- Define Common Interview Questions.
- Write one simple example of Common Interview Questions.
- Give one practical use of Common Interview Questions.

Chapter 33: Practice Programs

Practice without copying the answer.

Practice Programs



Problems

- Even/odd
- Prime
- Factorial
- Palindrome
- Reverse string
- Second largest distinct element
- Remove duplicates

Example Program

```
n=int(input('Enter number: '))
print('Even' if n%2==0 else 'Odd')
```

Output: Depends on input

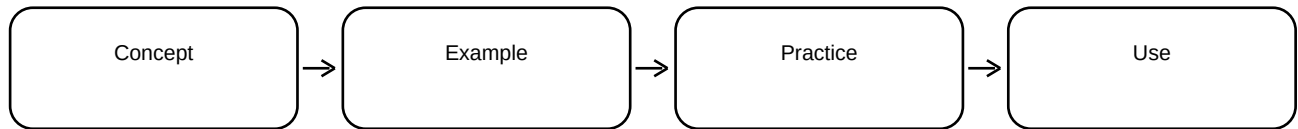
Practice Questions

- Define Practice Programs.
- Write one simple example of Practice Programs.
- Give one practical use of Practice Programs.

Chapter 34: Final Revision

Revise concepts by writing small programs and explaining them in your own words.

Final Revision



Checklist

- Syntax
- Data types
- Conditions
- Loops
- Collections
- Functions
- OOP
- Files
- NumPy/Pandas/Matplotlib

Example Program

```
print('Keep practicing Python!')
```

Output: Keep practicing Python!

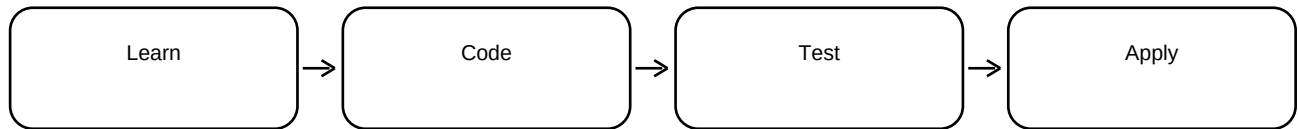
Practice Questions

- Define Final Revision.
- Write one simple example of Final Revision.
- Give one practical use of Final Revision.

Revision Page 1: Python Built-in Functions

print(), input(), len(), type(), int(), float(), str(), bool(), range(), sum(), min(), max(), sorted(), enumerate(), zip().

Python Built-in Functions



Example

```
nums=[5,2,9]
print(len(nums)); print(max(nums)); print(sorted(nums))
```

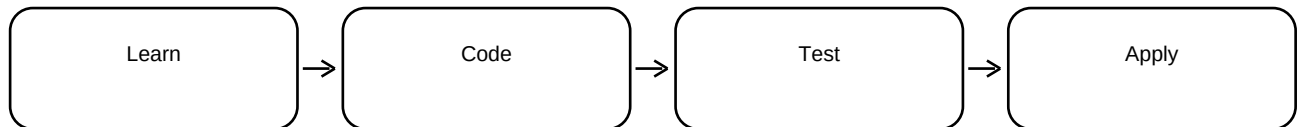
Quick practice

- Explain Python Built-in Functions in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 2: String Methods

upper(), lower(), strip(), split(), join(), replace(), find(), count(), startswith(), endswith().

String Methods



Example

```
text=' Python '  
print(text.strip().upper())
```

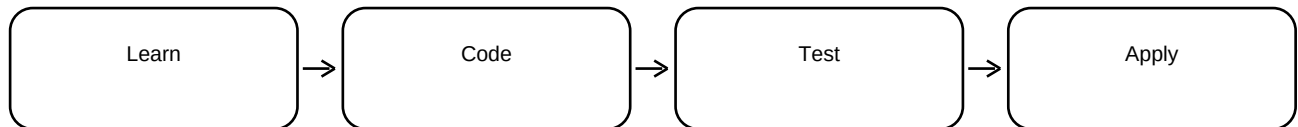
Quick practice

- Explain String Methods in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 3: List Methods

append(), extend(), insert(), remove(), pop(), clear(), index(), count(), sort(), reverse(), copy().

List Methods



Example

```
nums=[3,1,2]
nums.sort()
print(nums)
```

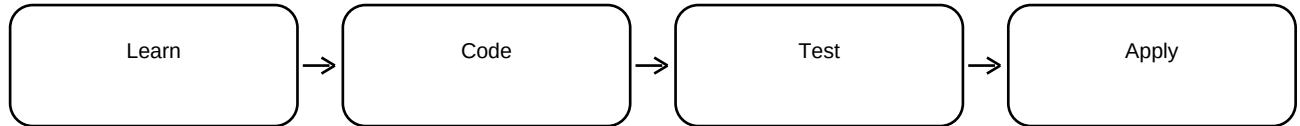
Quick practice

- Explain List Methods in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 4: Dictionary Methods

get(), keys(), values(), items(), update(), pop(), popitem(), clear(), copy().

Dictionary Methods



Example

```
d={'a':10}  
print(d.get('a'))
```

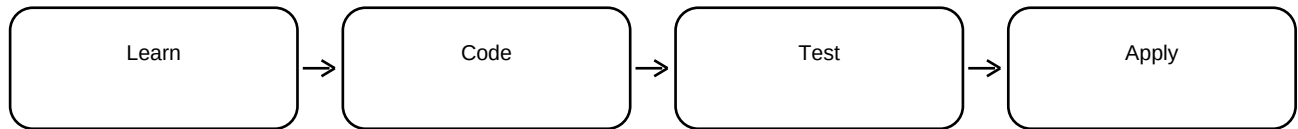
Quick practice

- Explain Dictionary Methods in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 5: Python Keywords

Common keywords include if, else, for, while, def, return, class, import, try, except, with, True, False, None, and, or, not, in, is, pass.

Python Keywords



Example

```
x=10
if x>5:
    print('Yes')
```

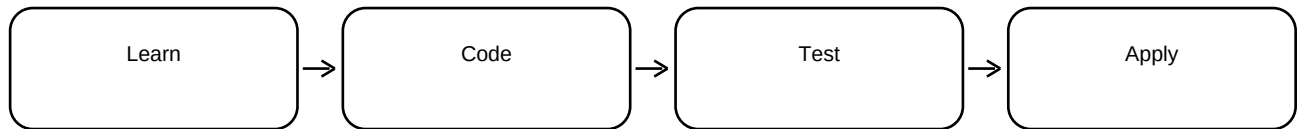
Quick practice

- Explain Python Keywords in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 6: Flowchart Thinking

A flowchart shows the order of steps. Example: Start → Input → Decision → Process → Output → End.

Flowchart Thinking



Example

```
n=10
if n%2==0:
    print('Even')
else:
    print('Odd')
```

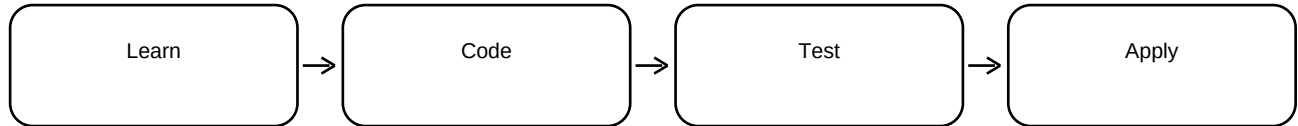
Quick practice

- Explain Flowchart Thinking in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 7: Algorithm Basics

An algorithm is a clear sequence of steps used to solve a problem.

Algorithm Basics



Example

```
numbers=[4,8,2]
largest=numbers[0]
for x in numbers[1:]:
    if x>largest: largest=x
print(largest)
```

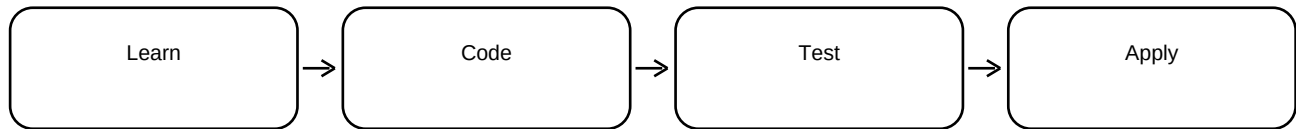
Quick practice

- Explain Algorithm Basics in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 8: Time Complexity Basics

$O(1)$ is constant time. $O(n)$ often comes from one pass through n items. $O(\log n)$ is common in binary search. $O(n^2)$ often comes from two nested loops.

Time Complexity Basics



Example

```
for x in range(5):  
    print(x)
```

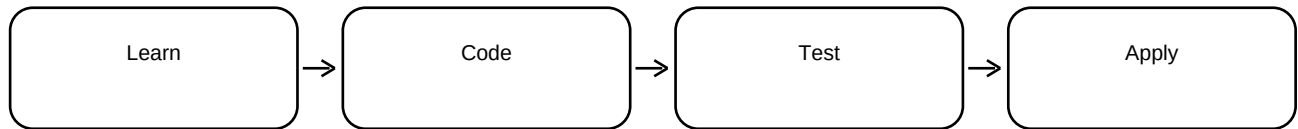
Quick practice

- Explain Time Complexity Basics in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 9: Space Complexity Basics

Space complexity describes extra memory. A few variables are often $O(1)$; an additional collection with n elements may be $O(n)$.

Space Complexity Basics



Example

```
numbers=[1, 2, 3]
copy=numbers.copy()
print(copy)
```

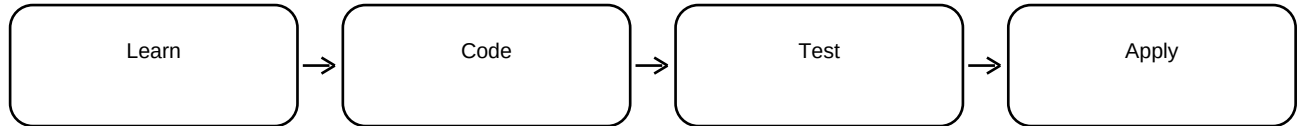
Quick practice

- Explain Space Complexity Basics in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 10: Linear Search

Linear search checks items one by one and works even when data is unsorted.

Linear Search



Example

```
nums=[5,8,2,9]
target=2
for i,x in enumerate(nums):
    if x==target:
        print(i)
        break
```

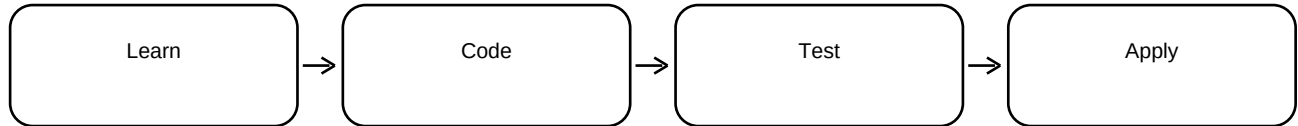
Quick practice

- Explain Linear Search in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 11: Binary Search

Binary search requires sorted data and repeatedly cuts the search area in half.

Binary Search



Example

```
nums=[1,3,5,7,9]
target=7
left,right=0,len(nums)-1
while left<=right:
    mid=(left+right)//2
    if nums[mid]==target: print(mid); break
    if nums[mid]<target: left=mid+1
    else: right=mid-1
```

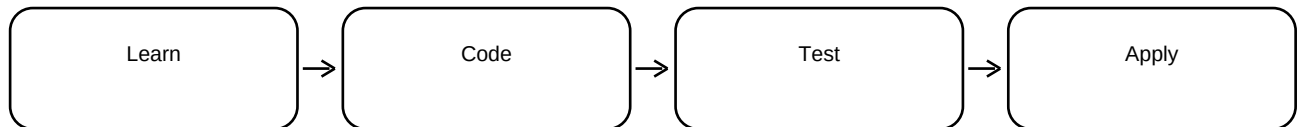
Quick practice

- Explain Binary Search in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 12: Sorting Basics

Sorting arranges values in order. Python provides `sorted()` and `list.sort()`.

Sorting Basics



Example

```
nums=[5,2,9,1]  
print(sorted(nums))
```

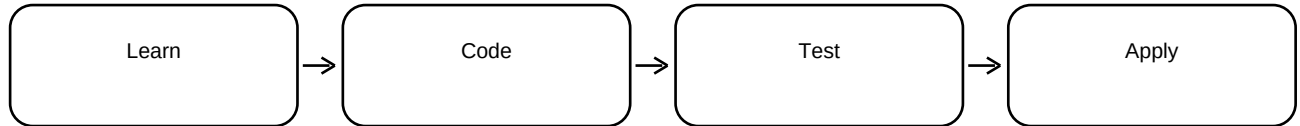
Quick practice

- Explain Sorting Basics in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 13: JSON Basics

JSON is a common text format for exchanging structured data.

JSON Basics



Example

```
import json
data='{"name": "Bhavya", "score": 90}'
obj=json.loads(data)
print(obj['name'])
```

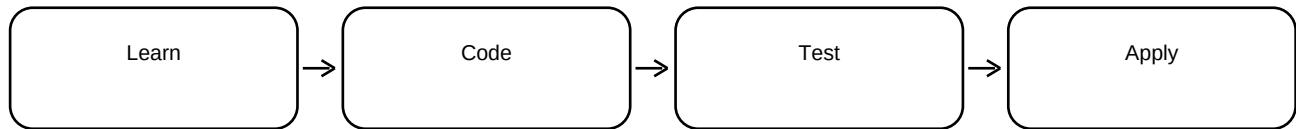
Quick practice

- Explain JSON Basics in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 14: APIs with Python

An API lets one program request data or services from another system. A common workflow is request → response → JSON → processing.

APIs with Python



Example

```
response_data={'temperature':28,'city':'Ongole'}  
print(response_data['temperature'])
```

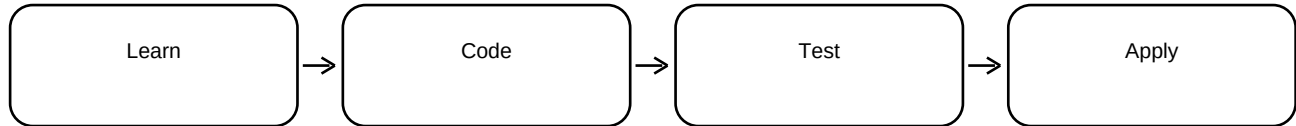
Quick practice

- Explain APIs with Python in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 15: Regular Expressions

Regular expressions are patterns for finding or validating text.

Regular Expressions



Example

```
import re
text='Email: student@example.com'
m=re.search(r'[\w.-]+@[\w.-]+',text)
print(m.group())
```

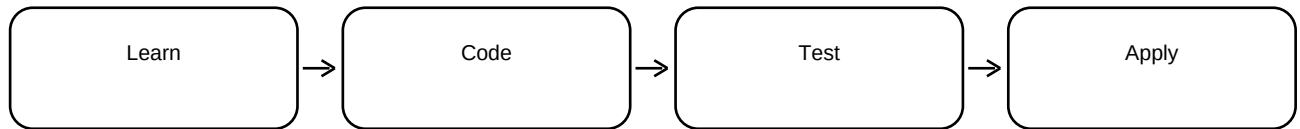
Quick practice

- Explain Regular Expressions in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 16: Virtual Environments

Virtual environments keep project dependencies separate. Create an environment, activate it, and install the required packages.

Virtual Environments



Example

```
print('Use a separate environment for each project.')
```

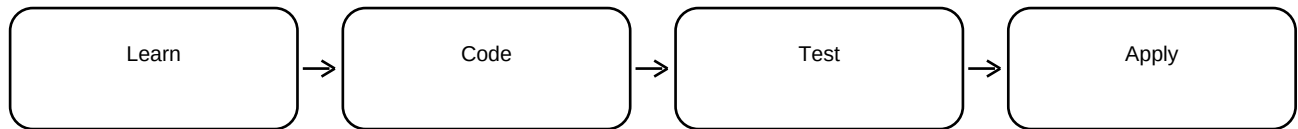
Quick practice

- Explain Virtual Environments in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 17: Testing Basics

Testing checks whether code behaves as expected. Assertions are useful for small checks.

Testing Basics



Example

```
def add(a,b): return a+b
assert add(2,3)==5
print('Test passed')
```

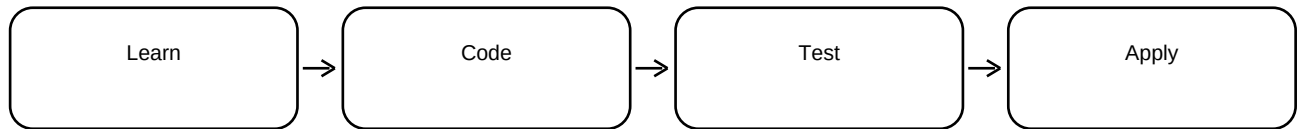
Quick practice

- Explain Testing Basics in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 18: Clean Code Checklist

Use meaningful names, small functions, consistent formatting, limited repetition, input validation, error handling, and tests.

Clean Code Checklist



Example

```
def calculate_total(price, tax):  
    return price + tax  
print(calculate_total(100,18))
```

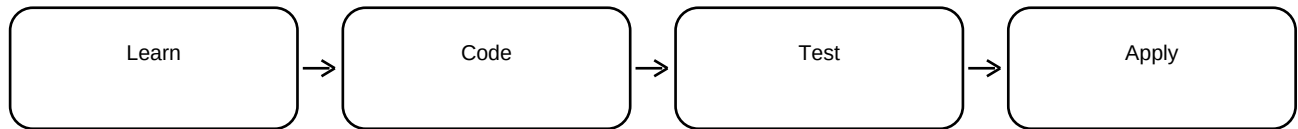
Quick practice

- Explain Clean Code Checklist in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 19: Exam Tips

For theory: definition → explanation → example → use. For programs: understand input → write logic → code → test with sample values.

Exam Tips



Example

```
print('Understand the logic before memorizing code.')
```

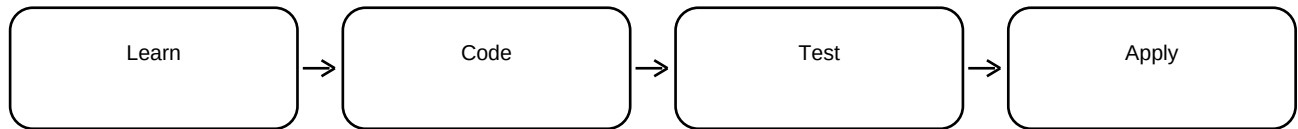
Quick practice

- Explain Exam Tips in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 20: Interview Revision

Practice explaining Python, variables, collections, functions, exceptions, OOP, NumPy, Pandas, and visualization in simple words.

Interview Revision



Example

```
print('Definition + Example + Use')
```

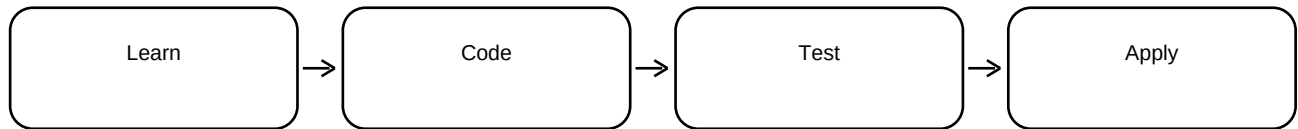
Quick practice

- Explain Interview Revision in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 21: Final Practice Test

Theory: Python features, list vs tuple, exception handling, OOP, DataFrame. Programs: even/odd, prime, factorial, palindrome, second largest.

Final Practice Test



Example

```
print('Practice before checking solutions.')
```

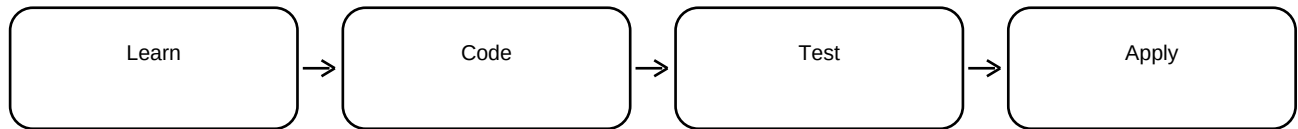
Quick practice

- Explain Final Practice Test in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.

Revision Page 22: 30-Day Learning Plan

Days 1–5 basics; 6–10 conditions and loops; 11–15 collections; 16–20 functions/OOP; 21–25 files and libraries; 26–30 mini project.

30-Day Learning Plan



Example

```
print('Day 1: Variables and data types')
```

Quick practice

- Explain 30-Day Learning Plan in your own words.
- Write one additional program related to this topic.
- Test your program with at least three different inputs.